

Notebook

February 8, 2026

1 RSA Least Significant Bit Oracle

Imagine, you have a side channel on the server receiving RSA ciphertexts, that allows you to get just the least significant bit of corresponding plaintext? Seems, like you wouldn't be able to retrieve much information, right? Wrong. Even leaking 1 bit can break the whole cryptosystem. Here is how. Remember, RSA is a homomorphic encryption, so:

$$\forall M_1, M_2 \in \mathbb{Z}_N^*, C_1 C_2 \bmod N = (M_1 M_2)^e \bmod N = M_1^e M_2^e \bmod N$$

Special case:

$$Enc(2M) = 2^e M^e \bmod N$$

We can use this property to recover the whole plaintext. Let's look, which least significant bit we'll receive if we multiply the plaintext M by 2. You could think, that it will always be 0, since the least significant bit of any integer multiplied by 2 is 0. However, we are multiplying modulo N , and if the resulting plaintext is greater or equal to N , we subtract N from it until it is less than N . What exactly happens if we multiply by 2? We know, that M is less than N initially:

$$M < N$$

So

$$2M < 2N$$

and we have 2 cases:

$$2M < N$$

and

$$2M > N$$

$2M \neq N$ since N is odd. In the case, where $2M < N$, we don't subtract N from the result of multiplication, so $LSB(2M \bmod N) = 0$. However, if $2M > N$, the resulting plaintext is actually $2M - N$, and since N is odd $LSB(2M \bmod N) = 1$. So, based on this, we can decide whether $2M < N$ or $2M > N$, which means $M < \lceil \frac{N}{2} \rceil$ or $M \geq \lceil \frac{N}{2} \rceil$. We just cut the domain of M by half. What's more, we can continue this process. If $LSB(4M) = 0$, then M is in the left half of the previously halved domain, alternatively if $LSB(4M) = 1$, then it is in the right half. We can continue this process either till we know M exactly, or M is in such small domain, that it would be easier to bruteforce it, than to leak bits.

You are given a server, that acts as a Least Significant Bit Oracle. You can submit ciphertexts to it and it will respond with the LSB of the plaintext. Recover the initial plaintext using this oracle. Good luck!

```
[ ]: import socket
import re
from Crypto.Util.number import inverse, long_to_bytes, bytes_to_long
class VulnServerClient:
    def __init__(self, show=True):
        """Initialization, connecting to server"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1343))
        if show:
            print (self.recv_until().decode())
    def recv_until(self, symb=b'\n>'):
        """Receive messages from server, by default till new prompt"""
        data=b''
        while True:
            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def get_public_key(self, show=True):
        """Receive public key from the server"""
        self.s.sendall('public\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        e=int(re.search('(?!<=e: )\d+', response).group(0))
        N=int(re.search('(?!<=N: )\d+', response).group(0))
        return (e,N)
    def get_ciphertext(self, show=True):
        """Receive ciphertext from the server"""
        self.s.sendall('ciphertext\n'.encode())
        response=self.recv_until().decode()
        if show:
            print (response)
        c=bytes_to_long(bytes.fromhex(re.search('(?!<=ciphertext: )[0-9a-f]+' , response).group(0)))
        return c
    def get_LSB_from_ciphertext(self, ciphertext, show=True):
        """Retrieve LSB for given ciphertext from the server"""
        if isinstance(ciphertext,int):
            ciphertext=long_to_bytes(ciphertext)
```

```

    if len(ciphertext)>256:
        print ('Ciphertext too long')
        return None
    if len (ciphertext)<256:
        ciphertext=bytes([0]*(256-len(ciphertext)))+ciphertext
        ciphertext_hex=ciphertext.hex()

    self.s.sendall(f'lsb {ciphertext_hex}\n'.encode())

    response=self.recv_until().decode()
    if show:
        print (response)
    lsb=int(re.search(r'(?<=lsb is: )\d',response).group(0))

    return lsb

def __del__(self):
    self.s.close()

```

```

[ ]: vs=VulnServerClient()
      (e,N)=vs.get_public_key()
      c=vs.get_ciphertext()
      print(c)
      print (vs.get_LSB_from_ciphertext(c, False))
      # I advise you to use the show=False option. 2000 requests with all
      ↪ that output is too many.

```

```

[ ]:

```